

```
In [ ]: ▶ # n1 = int(input()) # 10
# n2 = int(input()) # 20
# print(n1+n2)
# print (*range(2,10,2))
print (*range(2,10))
```

```
In [ ]: ▶ def isPrime(n):
    for fact in range(2,n):
        if n % fact == 0 :
            return False
    return True

# main function
n = int(input())
print ("Prime" if isPrime(n) else "Not Prime")
```

```
In [ ]: ▶ #Program to check whether the given is prime or not
def isPrime(n):
    for fact in range(2,n//2):
        if n % fact == 0 :
            return False
    return True

# main function
n = int(input())
print ("Prime" if isPrime(n) else "Not Prime")
```

```
In [ ]: ▶ #Program to check whether the given is prime or not
import math as m
def isPrime(n):
    for fact in range(2,int(m.sqrt(n))+1):
        if n % fact == 0 :
            return False
    return True

# main function
n = int(input())
print ("Prime" if isPrime(n) else "Not Prime")
```

```
In [ ]: """
All prime numbers are either 6N + 1 or 6N -1 except 2 & 3
but
All 6N+1 and 6N-1 are not prime numbers

"""
import math as m
def isPrime(n):
    if n == 1:
        return False
    if n > 3 and (n % 2 == 0 or n % 3 == 0):
        return False
    for fact in range(5,int(m.sqrt(n))+1,6):
        if n % fact == 0 or n % (fact+2)==0:
            return False
    return True

# main function
n = int(input())
print ("Prime" if isPrime(n) else "Not Prime")
```

```
In [ ]: print(*range(5,101, 6))
```

```
In [ ]: ## for index in range(n):
# #     data[index] = int(data[index])
# print (data[0]+data[1])
# print(data)

# list1 = [10,21,33,4,5,6]
# slicing
# [startIndex:endIndex:incrementValue]
# print(list1[:3]) # print first 3 values. It prints oth, 1st, 2nd index v
# print(list1[3:])
# print (list1[::-1])
# print (list1[3::-1])
```

```

In [ ]: ▶ #Program to sort the list using selection sort algorithm - greedy
def selectionSort(dataList, n):
    for i in range(n):
        mi = i
        # selection least value from the remaining list
        for j in range(i+1, n):
            if dataList[mi] > dataList[j]:
                mi = j
        #checking whether ith is least or not
        if i!=mi:
            dataList[mi], dataList[i] = dataList[i],dataList[mi]

n = int(input())
# data = list(map(int, input().split()))[:n]
data = []
for _ in range(n):
    data.append(int(input()))
selectionSort(data, n)
print(data)

```

```

In [ ]: ▶ list1 = ["Jegan", "Dhanya", "Nathish", "Yuvasri", "Sudharsan", "Isaimani",
# list1.sort(reverse=True) #descending order
# sortedList=sorted(list1)
sortedList=sorted(list1, reverse=True) #descending order
print(list1)
print(sortedList)

```

```

In [ ]: ▶ # Day - 2
#Program to print natural numbers using iterative method
n = int(input())
# for i in range(1,n+1):
#     print(i,end=" ")
print(*range(1,n+1))

```

```

In [2]: ▶ #Program to print natural numbers using recursive method
def printNatural(x):
    if x < 1:
        return
    print(x, end=" ")
    # recursive call
    printNatural(x-1)

# main function
n = int(input())
printNatural(n)

```

10

1 2 3 4 5 6 7 8 9 10

```
In [ ]: ▶ # def printNatural(5):
#         if 5 < 1:
#             return
#         printNatural(5-1)
#         print(5, end=" ")

# def printNatural(4):
#         if 4 < 1:
#             return
#         printNatural(4-1)
#         print(4, end=" ")

# def printNatural(3):
#         if 3 < 1:
#             return
#         printNatural(3-1)
#         print(3, end=" ")

# def printNatural(2):
#         if 2 < 1:
#             return
#         printNatural(2-1)
#         print(2, end=" ")

# def printNatural(1):
#         if 1 < 1:
#             return
#         printNatural(1-1)
#         print(1, end=" ")

# def printNatural(0):
#         if 0 < 1:
#             return
#         printNatural(0-1)
#         print(0, end=" ")
```

```
In [4]: ▶ #Program to demonstrate double recursion
def printf(n):
    #base case
    if n < 1:
        return
    printf(n-1) # head recursion
    print(n, end=" ")
    printf(n-1) # tail recursion

# main function
n = 3
printf(n)
```

1 2 1 3 1 2 1

```
In [16]: ▶ list1 = [1,2]
print(list1.__sizeof__())
list1.append(3)
print(list1.__sizeof__())
list1.append(4)
print(list1.__sizeof__())
list1.append(5)
print(list1.__sizeof__())
list1.append(5)
print(list1.__sizeof__())
list1.append(6)
print(list1.__sizeof__())
list1.append(7)
print(list1.__sizeof__())
list1.append(8)
print(list1.__sizeof__())
list1.append(9)
print(list1.__sizeof__())
list1.pop()
print(list1.__sizeof__())
```

```
56
104
104
104
104
104
104
168
168
168
```

```
In [29]: ▶ class Node:
    def __init__(self, value):
        self.data = value
        self.next = None

class LinkedList:
    def __init__(self):
        self.root = None
    def addFirst(self, value):
        newNode = Node(value)
        newNode.next = self.root
        self.root = newNode
    def addLast(self, value):
        newNode = Node(value)
        if self.root == None:
            self.root = newNode
        else:
            temp = self.root
            while temp.next != None:
                temp = temp.next # move the pointer to the next node
            temp.next = newNode

    def deleteFirst(self):
        # checking whether the list empty or not
        if self.root != None:
            # list is not empty
            self.root = self.root.next

    def deleteLast(self):
        pass

    def printList(self):
        if self.root == None:
            print("[]")
        else:
            temp = self.root;
            print("[", end="")
            while temp.next != None:
                print('"' + temp.data + '"', end=", ", sep="")
                temp = temp.next
            print('"' + temp.data + '"', end="]", sep="")

myList = LinkedList()
myList.addFirst("Hari")
myList.addFirst("Raj")
myList.addFirst("Jai")
myList.addLast("Subash")
myList.addFirst("Javid")
myList.printList()
myList.deleteFirst()
myList.printList()
```

```
["Javid", "Jai", "Raj", "Hari", "Subash"]
["Jai", "Raj", "Hari", "Subash"]
```

```
In [36]: ▶ #Program to implement coin change problem - greedy approach
amt = int(input()) # amount
n = int(input()) # number of denominations
denoms = list(map(int, input().split()))
denoms.sort(reverse=True)
totalCoins = 0
changeDetails = {}
for coin in denoms:
    if amt >= coin:
        cur = amt // coin
        changeDetails[coin] = cur
        totalCoins += cur
        amt = amt % coin # amt %= coin
    if amt == 0:
        break
if amt == 0:
    print(totalCoins)
    print (changeDetails)
else:
    print(-1)
```

```
1360
5
50 200 100 10 500
6
{500: 2, 200: 1, 100: 1, 50: 1, 10: 1}
```

```
In [6]: ▶ # Day - 3
# Program to implement activity selection problem - greedy
n = int(input())
activities = []
for _ in range(n):
    activities.append(list(map(int, input().split())))
# sorting the activities by its ending time
result = [] # to store participated activities
result.append(activities[0])
activities.sort(key=lambda activity : activity[1])
i = 0
while i < n:
    for j in range(i+1, n):
        if activities[j][0] >= activities[i][1]:
            result.append(activities[j])
            i = j;
    i+=1
print(result)
```

```
11
1 4
3 5
0 6
5 7
3 9
5 9
6 10
8 11
6 12
2 14
12 16
[[1, 4], [5, 7], [8, 11], [12, 16]]
```

```
In [16]: ▶ #Program to implement fractional knapsack problem
bagSize = int(input())
n = int(input())
products = []
for _ in range(n):
    profit, weight = list(map(int,input().split()))
    ratio = profit / weight
    products.append([ratio,profit, weight])
products.sort(reverse=True)
maxProfits = 0
for r,p,w in products:
    if bagSize >= w:
        maxProfits += p
        bagSize-=w
    else:
        maxProfits += bagSize * r
        bagSize = 0
print(maxProfits)
```

```
15
5
2 1
28 4
25 5
18 3
9 3
80.0
```

```
In [11]: ▶ # profit, weight = list(map(int,input().split()))
# print (profit, weight)
l1 = [1,2,3]
x,y,z = l1
print(x,y,z)
```

```
1 2 3
```